

Serial Peripheral Interface

BY THE SPIES TEAM

Youssef Nasser
Haneen Ayman
Maya Lasheen

RTL Code

SPI Module

```
module spi_slave_interface (  
    input clk, rst_n, SS_n,  
    input MOSI,  
    input [7:0] tx_data,  
    input tx_valid,  
  
    output MISO,  
    output reg [9:0] rx_data,  
    output rx_valid  
);  
  
    // States  
    parameter IDLE      = 3'd0;  
    parameter CHK_CMD   = 3'd1;  
    parameter READ_ADD  = 3'd2;  
    parameter READ_DATA = 3'd3;  
    parameter WRITE     = 3'd4;  
  
    reg [2:0] cs, ns;  
  
    reg flag; // distinguishes READ_ADD / READ_DATA  
    reg [3:0] rx_counter; // counts bits for serial to parallel conversion  
and vice versa  
    reg [2:0] tx_counter; // counts bits for serial to parallel conversion  
and vice versa  
    reg [7:0] tx_shift; // temporary register for MISO  
  
    // Next State Logic  
    always @(*) begin  
        case (cs)  
            IDLE:  
                if (~SS_n)  
                    ns = CHK_CMD;  
                else  
                    ns = cs;  
  
            CHK_CMD:  
                if (~SS_n) begin  
                    if (MOSI) begin  
                        if (flag)  
                            ns = READ_DATA;  
                    end  
                end  
        endcase  
    end
```

```

        else
            ns = READ_ADD;
        end else
            ns = WRITE;
        end else
            ns = IDLE;

        READ_ADD, READ_DATA, WRITE:
            if (SS_n)
                ns = IDLE;
            else
                ns = cs;
        endcase

    end

    // State Memory
    always @(posedge clk or negedge rst_n) begin
        if (~rst_n)
            cs <= IDLE;
        else
            cs <= ns;
    end

    assign rx_valid = (rx_counter == 10) && (cs == WRITE || cs ==
    READ_ADD);
    assign MISO = tx_shift[7];

    always @(posedge clk or negedge rst_n) begin
        if (~rst_n) begin
            rx_data <= 0;
            rx_counter <= 0;
            tx_counter <= 0;
            tx_shift <= 0;
            flag <= 0;
        end else begin

            if (SS_n) begin
                rx_counter <= 0;
                tx_counter <= 0;
            end else begin
                case (cs)
                    CHK_CMD, WRITE, READ_ADD, READ_DATA: begin

```

```

        if (rx_counter == 10) begin
            rx_data <= 0;
            rx_counter <= 0;
        end else begin
            rx_data <= {rx_data[8:0], MOSI};
            rx_counter <= rx_counter + 1;
        end
        if (cs == READ_ADD)
            flag <= 1;

        if (cs == READ_DATA) begin
            if (tx_valid) begin
                tx_shift <= tx_data;
            end else begin
                tx_shift <= {tx_shift[6:0], 1'b0};
            end

            if (tx_counter == 7) begin
                tx_counter <= 0;
            end else begin
                tx_counter <= tx_counter + 1;
            end

            flag <= 0;
        end

    end

    default: begin
        rx_counter <= 0;
        tx_counter <= 0;
        rx_data <= 0;
    end

endcase
end
end
end
endmodule

```

RAM Module

```
module mem #(parameter ADDR_SIZE = 8, parameter MEM_DEPTH = 256) (  
    input clk, rst_n,  
    input [ADDR_SIZE+1:0] din,  
    input rx_valid,  
    output reg [ADDR_SIZE-1:0] dout,  
    output reg tx_valid  
);  
  
    // din first 2 bits indicate the op.  
    // the remaining din bits is equal to the address and as the word  
size.  
    // so if ADDR_SIZE = 8, din size should be 10, i.e., 9 downto 0.  
    // and the word size would be 7 downto 0 too, i.e, [ADDR_SIZE-1:0]  
  
    reg [ADDR_SIZE-1:0] mem_array [0:MEM_DEPTH-1];  
    reg [ADDR_SIZE-1:0] write_addr;  
    reg [ADDR_SIZE-1:0] read_addr;  
  
    always @(posedge clk or negedge rst_n) begin  
        if (~rst_n) begin  
            read_addr <= 0;  
            write_addr <= 0;  
            dout <= 0;  
            tx_valid <= 0;  
        end else begin  
            if (rx_valid) begin  
                case (din[ADDR_SIZE+1:ADDR_SIZE])  
                    2'b00: begin  
                        write_addr <= din[ADDR_SIZE-1:0];  
                        tx_valid <= 0;  
                    end  
                    2'b01: begin  
                        mem_array[write_addr] <= din[ADDR_SIZE-1:0];  
                        tx_valid <= 0;  
                    end  
                    2'b10: begin  
                        read_addr <= din[ADDR_SIZE-1:0];  
                        tx_valid <= 0;  
                    end  
                    default: begin  
                        dout <= 0;  
                        tx_valid <= 0;  
                    end  
                end  
            end  
        end  
    end
```

```

        endcase
    end else if (din[ADDR_SIZE+1:ADDR_SIZE] == 2'b11) begin
        dout <= mem_array[read_addr];
        tx_valid <= 1;
    end else begin
        dout <= 0;
        tx_valid <= 0;
    end
end
end
end
endmodule

```

Wrapper Module

```

module wrapper #(parameter ADDR_SIZE = 8, parameter MEM_DEPTH = 256) (
    input clk, rst_n, SS_n,
    input MOSI,
    output MISO
);

    wire [7:0] tx_data;
    wire tx_valid;
    wire [9:0] rx_data;
    wire rx_valid;

    spi_slave_interface spi (.clk(clk), .rst_n(rst_n), .SS_n(SS_n),
        .MOSI(MOSI), .tx_data(tx_data), .tx_valid(tx_valid), .MISO(MISO),
        .rx_data(rx_data), .rx_valid(rx_valid));

    mem #(.ADDR_SIZE(ADDR_SIZE), .MEM_DEPTH(MEM_DEPTH)) RAM (.clk(clk),
        .rst_n(rst_n), .din(rx_data), .rx_valid(rx_valid), .dout(tx_data),
        .tx_valid(tx_valid));

endmodule

```

Testbench Code

```
module spi_tb ();

parameter MEM_DEPTH = 256;
parameter ADDR_SIZE = 8;

reg clk, rst_n, MOSI, SS_n;
wire MISO;

wrapper #(.MEM_DEPTH(MEM_DEPTH), .ADDR_SIZE(ADDR_SIZE)) dut (.clk(clk),
.rst_n(rst_n), .MOSI(MOSI), .SS_n(SS_n), .MISO(MISO));

initial begin
    clk = 0;
    forever
        #1 clk = ~clk;
end

//RAM Write Command - Write Address
initial begin
    rst_n = 0;
    SS_n = 1;
    MOSI = 0;
    @(negedge clk);
    rst_n = 1;

    SS_n = 0; // cs = IDLE, ns = CHK_CMD
    @(negedge clk);
    MOSI = 0; // cs = CHK_CMD, ns = WRITE, rx_counter <= 1, rx_data <=
00_0000_0000 = 000
    @(negedge clk);
    MOSI = 0; // cs = WRITE, rx_counter <= 2, rx_data <= 00_0000_0000 =
000
    @(negedge clk);
    MOSI = 1; // cs = WRITE, rx_counter <= 3, rx_data <= 00_0000_0001 =
001
    @(negedge clk);
    MOSI = 1; // cs = WRITE, rx_counter <= 4, rx_data <= 00_0000_0011 =
003
    @(negedge clk);
    MOSI = 1; // cs = WRITE, rx_counter <= 5, rx_data <= 00_0000_0111 =
007
    @(negedge clk);
```

```

    MOSI = 1; // cs = WRITE, rx_counter <= 6, rx_data <= 00_0000_1111 =
00f
    @(negedge clk);
    MOSI = 1; // cs = WRITE, rx_counter <= 7, rx_data <= 00_0001_1111 =
01f
    @(negedge clk);
    MOSI = 1; // cs = WRITE, rx_counter <= 8, rx_data <= 00_0011_1111 =
03f
    @(negedge clk);
    MOSI = 0; // cs = WRITE, rx_counter <= 9, rx_data <= 00_0111_1110 =
07e
    @(negedge clk);
    MOSI = 1; // cs = WRITE, rx_counter <= 10, rx_data <= 00_1111_1101 =
0fd
    @(negedge clk);

    SS_n = 1;
    @(negedge clk); // Wait 1 cycle for write_addr to get updated

//RAM Write Command - Write Data
    SS_n = 0; // ns = CHK_CMD
    @(negedge clk);
    MOSI = 0; // rx_data <= 00_0000_0000 = 000
    @(negedge clk);
    MOSI = 1; // rx_data <= 00_0000_0001 = 001
    @(negedge clk);
    MOSI = 1; // rx_data <= 00_0000_0011 = 003
    @(negedge clk);
    MOSI = 0; // rx_data <= 00_0000_0110 = 006
    @(negedge clk);
    MOSI = 1; // rx_data <= 00_0000_1101 = 00d
    @(negedge clk);
    MOSI = 1; // rx_data <= 00_0001_1011 = 01b
    @(negedge clk);
    MOSI = 0; // rx_data <= 00_0011_0110 = 036
    @(negedge clk);
    MOSI = 1; // rx_data <= 00_0110_1101 = 06d
    @(negedge clk);
    MOSI = 0; // rx_data <= 00_1101_1010 = 0da
    @(negedge clk);
    MOSI = 1; // rx_data <= 01_1011_0101 = 0b5
    @(negedge clk);

    SS_n = 1;

```



```

    @(negedge clk); // Wait 1 cycle for memory to get updated

//RAM Write Command - Read Address
    SS_n = 0; // ns = CHK_CMD
    @(negedge clk);
    MOSI = 1; // rx_data <= 00_0000_0001 = 001
    @(negedge clk);
    MOSI = 0; // rx_data <= 00_0000_0010 = 002
    @(negedge clk);
    MOSI = 1; // rx_data <= 00_0000_0101 = 005
    @(negedge clk);
    MOSI = 1; // rx_data <= 00_0000_1011 = 00b
    @(negedge clk);
    MOSI = 1; // rx_data <= 00_0001_0111 = 017
    @(negedge clk);
    MOSI = 1; // rx_data <= 00_00010_1111 = 02f
    @(negedge clk);
    MOSI = 1; // rx_data <= 00_0101_1111 = 05f
    @(negedge clk);
    MOSI = 1; // rx_data <= 00_1011_1111 = 0bf
    @(negedge clk);
    MOSI = 0; // rx_data <= 01_0111_1110 = 17e
    @(negedge clk);
    MOSI = 1; // rx_data <= 10_1111_1101= 2fd
    @(negedge clk);

    SS_n = 1;
    @(negedge clk); // Wait 1 cycle for read_addr to get updated

//RAM Write Command - Read Data
    SS_n = 0;
    @(negedge clk);
    MOSI = 1;
    @(negedge clk);
    MOSI = 1;
    @(negedge clk);
    MOSI = $random;
    @(negedge clk);
    MOSI = $random;
    @(negedge clk);
    MOSI = $random;
    @(negedge clk);
    MOSI = $random;
    @(negedge clk);

```

```
MOSI = $random;
@(negedge clk);
MOSI = $random;
@(negedge clk);
MOSI = $random;
@(negedge clk);
MOSI = $random;
@(negedge clk);

#20 $stop;
end

initial
    $monitor ("MOSI= %d , SS_n=%d, MISO=%d, clk=%d, rst_n=%d",MOSI, SS_n,
MISO, clk, rst_n);
endmodule
```

Do File

```
vlib work
vlog spi_slave_interface.v mem.v wrapper.v SPI_tb.v
vsim -voptargs=+acc work.spi_tb
```

```
add wave *
add wave -position insertpoint \
sim:/spi_tb/dut/spi/rx_counter \
sim:/spi_tb/dut/spi/rx_data \
sim:/spi_tb/dut/spi/rx_valid \
sim:/spi_tb/dut/spi/tx_counter \
sim:/spi_tb/dut/spi/tx_data \
sim:/spi_tb/dut/spi/tx_valid
```

```
add wave -position insertpoint \
sim:/spi_tb/dut/RAM/write_addr
add wave -position insertpoint \
sim:/spi_tb/dut/RAM/read_addr
```

```
add wave -position insertpoint \
sim:/spi_tb/dut/spi/flag
```

```
add wave -position insertpoint \
sim:/spi_tb/dut/spi/cs
```

```
add wave -position insertpoint \
sim:/spi_tb/dut/spi/tx_shift
```

```
add wave -position insertpoint \
sim:/spi_tb/dut/RAM/mem_array
```

```
run -all
```

Test Procedure

1. Initialization

The test begins by asserting a reset to initialize the DUT to a known state.

2. WRITE ADDRESS Operation

The WRITE ADDRESS operation is tested first. The first two MOSI bits are set to 2'b00, which serves as the command identifier for this operation. The remaining 8 bits are configured to 0xFD. The objective of this step is to write the address 0xFD into the write_addr register, which will be used as the target address in the subsequent WRITE DATA operation.

3. WRITE DATA Operation

Next, the WRITE DATA operation is validated. The first two MOSI bits are set to 2'b01 to indicate a WRITE DATA command. The remaining 8 bits are assigned a value of 0xB5. The goal here is to write the data 0xB5 into the memory location previously stored as 0xFD.

4. READ Operation

The READ operation is tested in two phases:

4.1 READ ADDRESS Phase

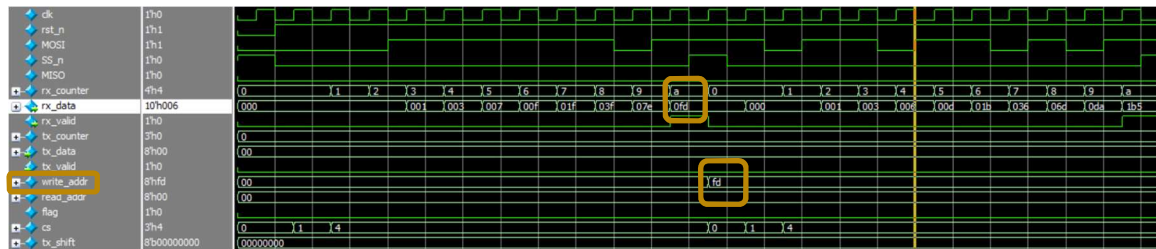
The first two MOSI bits are set to 2'b10 to denote the READ ADDRESS command. The remaining 8 bits are assigned 0xFD, specifying the memory address to read from.

4.2 READ DATA Phase

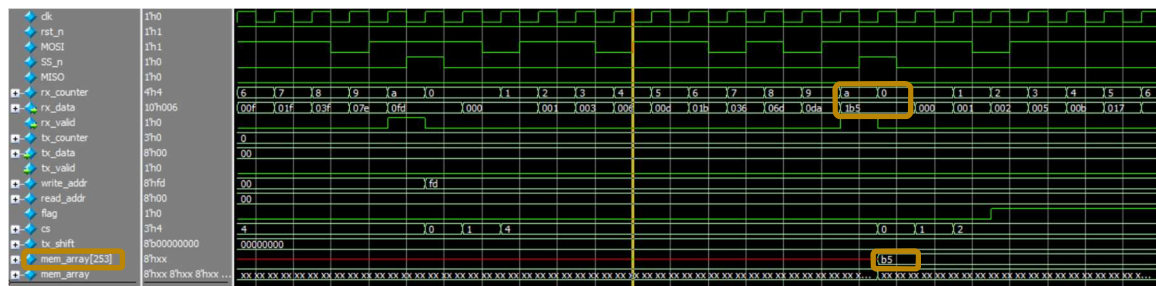
The first two MOSI bits are set to 2'b11 for the READ DATA command. The remaining 8 bits are randomized, as they are "don't care" values for this operation. The MISO output is monitored and compared against the serialized version of 0xB5 to verify correct read functionality.

Simulation Results – Waveform Snippets

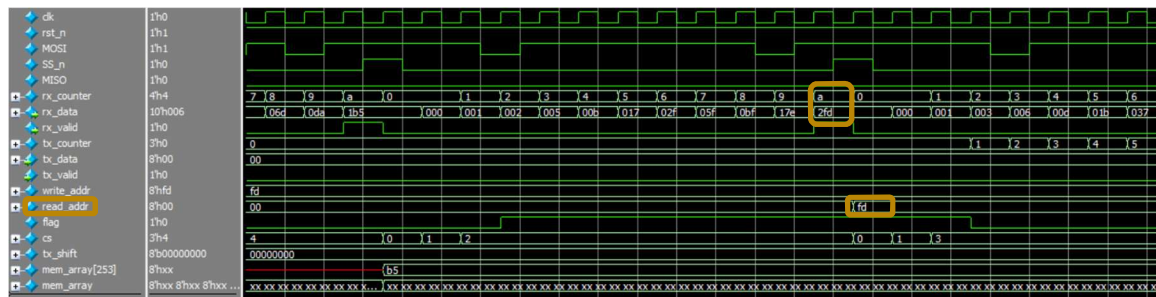
WRITE ADDRESS Operation



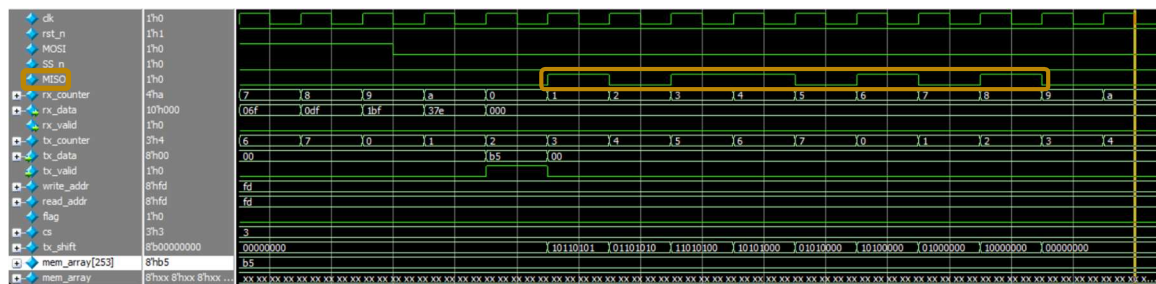
WRITE DATA Operation



READ ADDRESS Operation



READ DATA Operation



Linting Results

Lint Summary

Name	Count
Open(uninspected, pending, bug)	8
Warning	1
seq_block_has_duplicate_assign	1
Info	7
parameter_name_duplicate	2
line_char_large	2
multi_ports_in_single_line	3

Lint Checks

Severity	Status	Check	Alias	Message	Module
Warning	Fixed	seq_block_has_duplicate_assign		Signal is assigned more than once in a sequential block. Signal flag, Module spi_slave_interface, File C:/QuestaLint/spi_slave_i...	Rtl
Warning	Fixed	parameter_name_duplicate		Same parameter name is used in more than one module. Parameter ADDR_SIZE, Total count 2. First module: Module mem, Fi...	mem
Warning	Fixed	parameter_name_duplicate		Same parameter name is used in more than one module. Parameter MEM_DEPTH, Total count 2. First module: Module mem, ...	mem
Warning	Fixed	line_char_large		Line has more characters than the specified limit. Current Count 163, Specified Limit 110, File C:/QuestaLint/wrapper.v, Line 12.	none
Warning	Fixed	line_char_large		Line has more characters than the specified limit. Current Count 149, Specified Limit 110, File C:/QuestaLint/wrapper.v, Line 14.	none
Warning	Fixed	multi_ports_in_single_line		Multiple ports are declared in one line. Module mem, File C:/QuestaLint/mem.v, Line 2.	mem
Warning	Fixed	multi_ports_in_single_line		Multiple ports are declared in one line. Module spi_slave_interface, File C:/QuestaLint/spi_slave_interface.v, Line 2.	spi_slave_inte...
Warning	Fixed	multi_ports_in_single_line		Multiple ports are declared in one line. Module wrapper, File C:/QuestaLint/wrapper.v, Line 2.	wrapper

Constraints File

Clock signal

```
set_property -dict { PACKAGE_PIN W5 IOSTANDARD LVCMOS33 } [get_ports clk]
```

```
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]
```

Switches

```
set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports {SS_n}]
```

```
set_property -dict { PACKAGE_PIN V16 IOSTANDARD LVCMOS33 } [get_ports {MOSI}]
```

LEDs

```
set_property -dict { PACKAGE_PIN U16 IOSTANDARD LVCMOS33 } [get_ports {MISO}]
```

##Buttons

```
set_property -dict { PACKAGE_PIN U18 IOSTANDARD LVCMOS33 } [get_ports rst_n]
```

Configuration options, can be used for all designs

```
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

```
set_property CFGBVS VCCO [current_design]
```

```
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
```

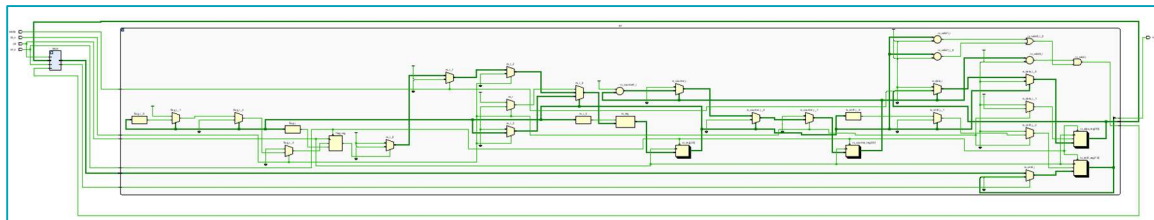
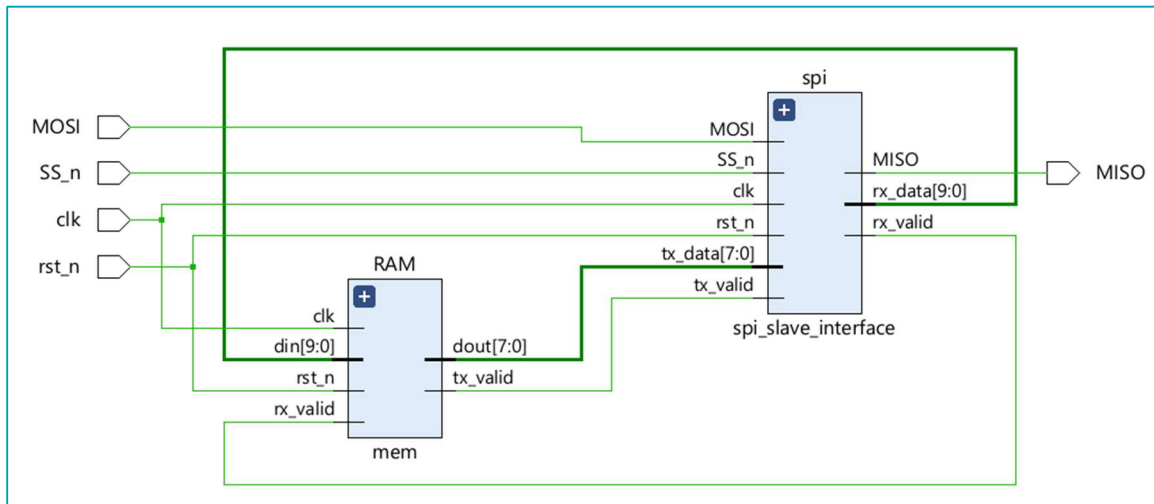
```
set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
```

```
set_property CONFIG_MODE SPIx4 [current_design]
```

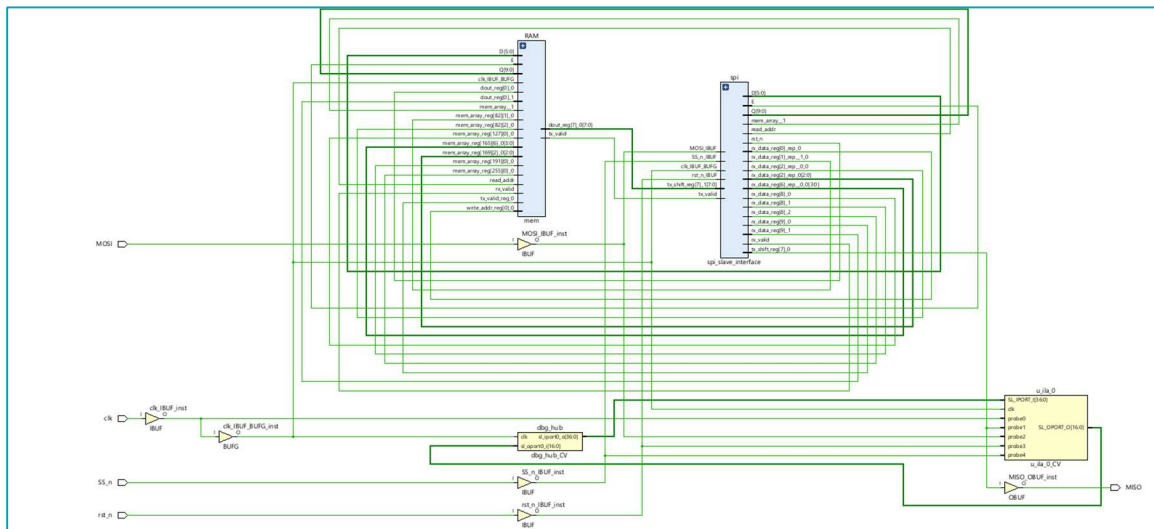
Synthesis & Implementation Results

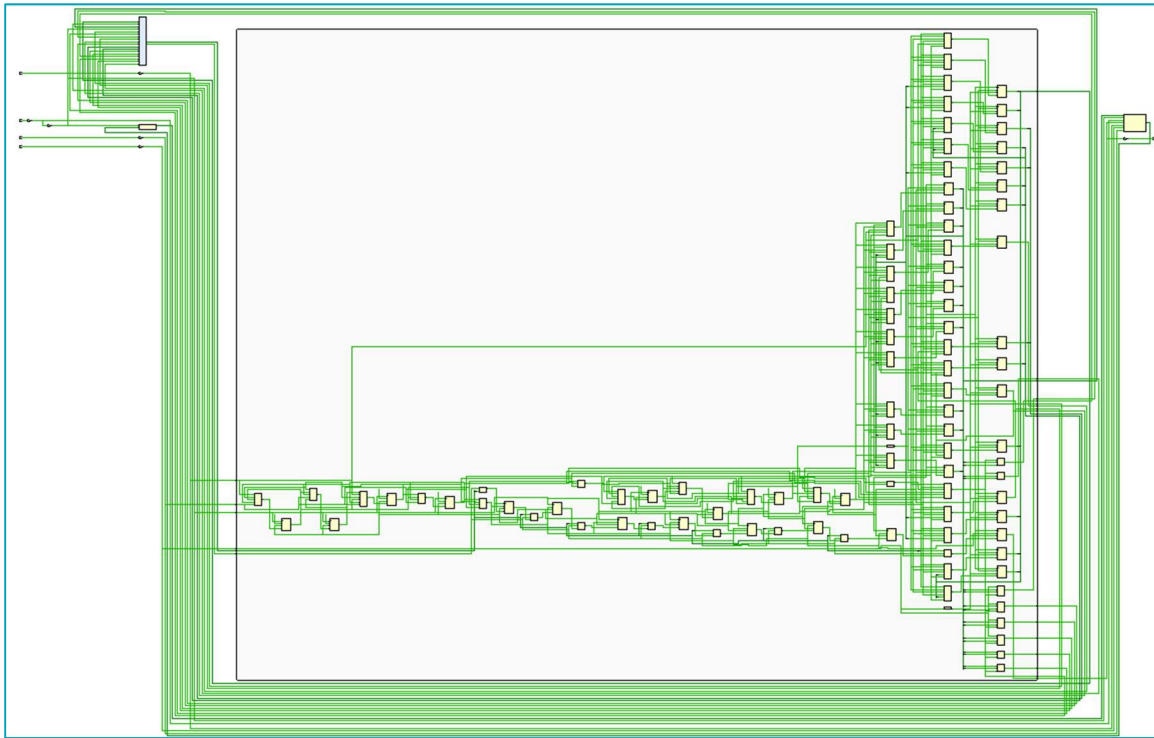
Grey Encoding

- Post-Elaboration Schematic



- Post-Synthesis Schematic





- Synthesis Report

State	New Encoding	Previous Encoding
IDLE	000	000
CHK_CMD	001	001
READ_DATA	011	011
READ_ADD	010	010
WRITE	111	100

INFO: [Synth 8-3354] encoded FSM with state register 'cs_reg' using encoding 'gray' in module 'spi_slave_interface'

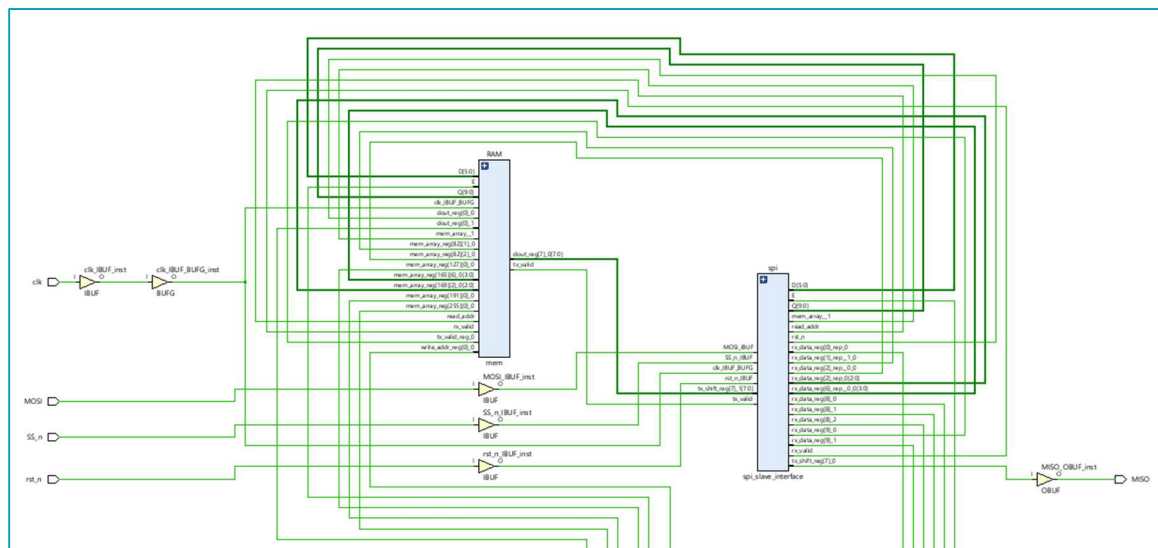
- Post-Synthesis Timing Report

Design Timing Summary		
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.753 ns	Worst Hold Slack (WHS): 0.134 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 4222	Total Number of Endpoints: 4222	Total Number of Endpoints: 2131
All user specified timing constraints are met.		

- Critical Path

Critical path is from the clock pin of the memory array register in the RAM module to the data pin of the RAM module.

Name	Slack	Levels	Routes	High Fanout	From	To	Logic Delay	Total Delay	Net Delay
Path 1	5.753	6	7	1	RAM/mem_array_reg[247][0]/C	RAM/dout_reg[0]/D	1.962	4.096	2.134
Path 2	5.753	6	7	1	RAM/mem_array_reg[247][1]/C	RAM/dout_reg[1]/D	1.962	4.096	2.134
Path 3	5.753	6	7	1	RAM/mem_array_reg[247][2]/C	RAM/dout_reg[2]/D	1.962	4.096	2.134
Path 4	5.753	6	7	1	RAM/mem_array_reg[247][3]/C	RAM/dout_reg[3]/D	1.962	4.096	2.134
Path 5	5.753	6	7	1	RAM/mem_array_reg[247][4]/C	RAM/dout_reg[4]/D	1.962	4.096	2.134
Path 6	5.753	6	7	1	RAM/mem_array_reg[247][5]/C	RAM/dout_reg[5]/D	1.962	4.096	2.134
Path 7	5.753	6	7	1	RAM/mem_array_reg[247][6]/C	RAM/dout_reg[6]/D	1.962	4.096	2.134
Path 8	5.753	6	7	1	RAM/mem_array_reg[247][7]/C	RAM/dout_reg[7]/D	1.962	4.096	2.134
Path 9	5.929	4	5	64	spi/rx_counter_reg[2]/C	RAM/mem_array_reg[0][0]/CE	1.149	3.689	2.540
Path 10	5.929	4	5	64	spi/rx_counter_reg[2]/C	RAM/mem_array_reg[0][1]/CE	1.149	3.689	2.540



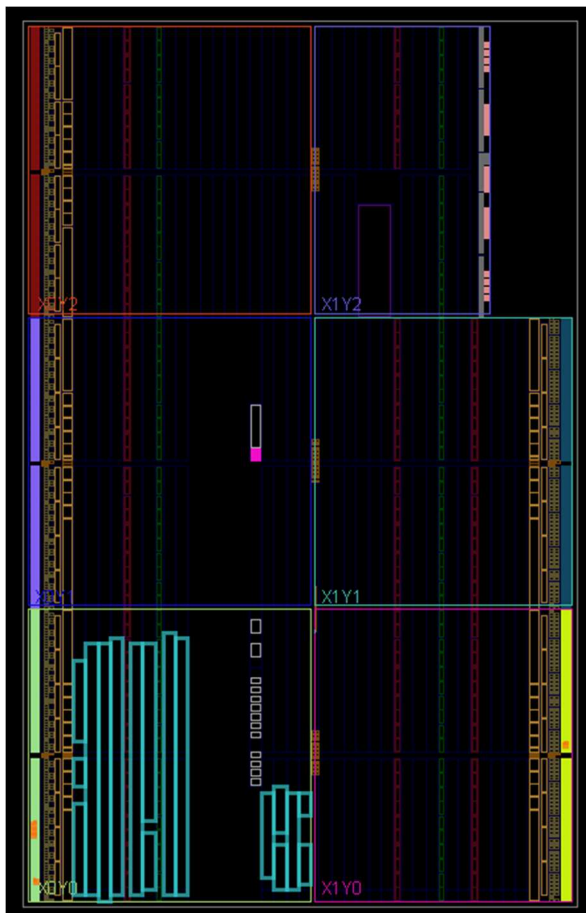
- Post-Implementation Utilization Report

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	F8 Muxes (8150)	Slice (8150)	LUT as Logic (20800)	Bonded IOB (106)	BUFGCTRL (32)
▼ N wrapper	893	2130	272	128	729	893	5	1
RAM (mem)	848	2088	272	128	719	848	0	0
spi (spi_slave_interface)	45	42	0	0	14	45	0	0

- Post-Implementation Timing Report

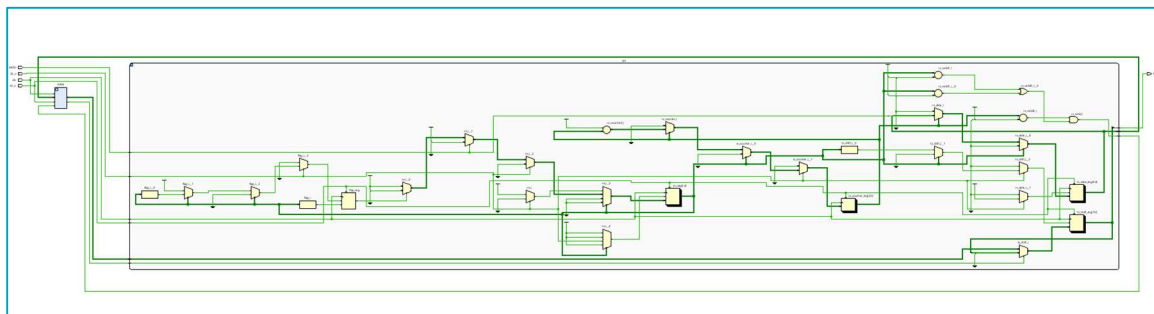
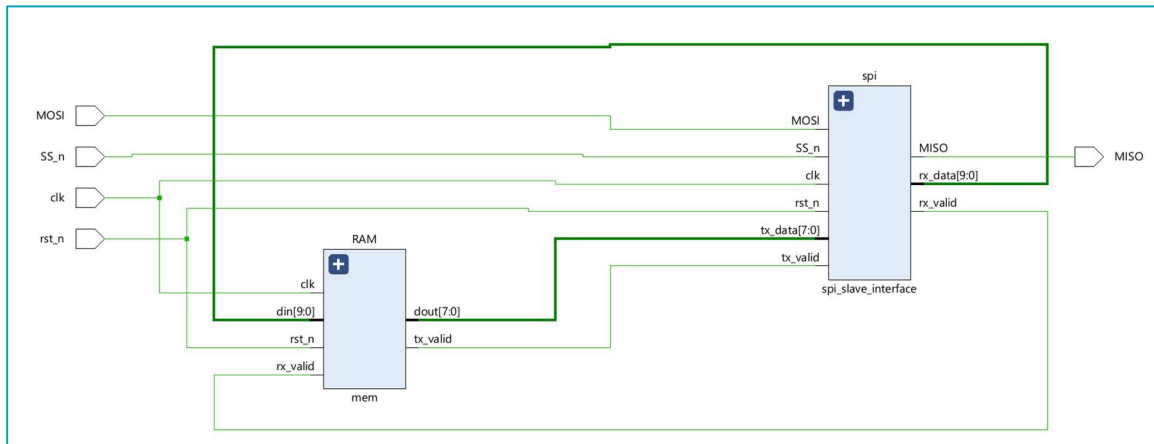
Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 2.829 ns		Worst Hold Slack (WHS): 0.007 ns		Worst Pulse Width Slack (WPWS): 3.750 ns	
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 8025		Total Number of Endpoints: 8009		Total Number of Endpoints: 4255	
All user specified timing constraints are met.					

- Post-Implementation FPGA Device Snippet

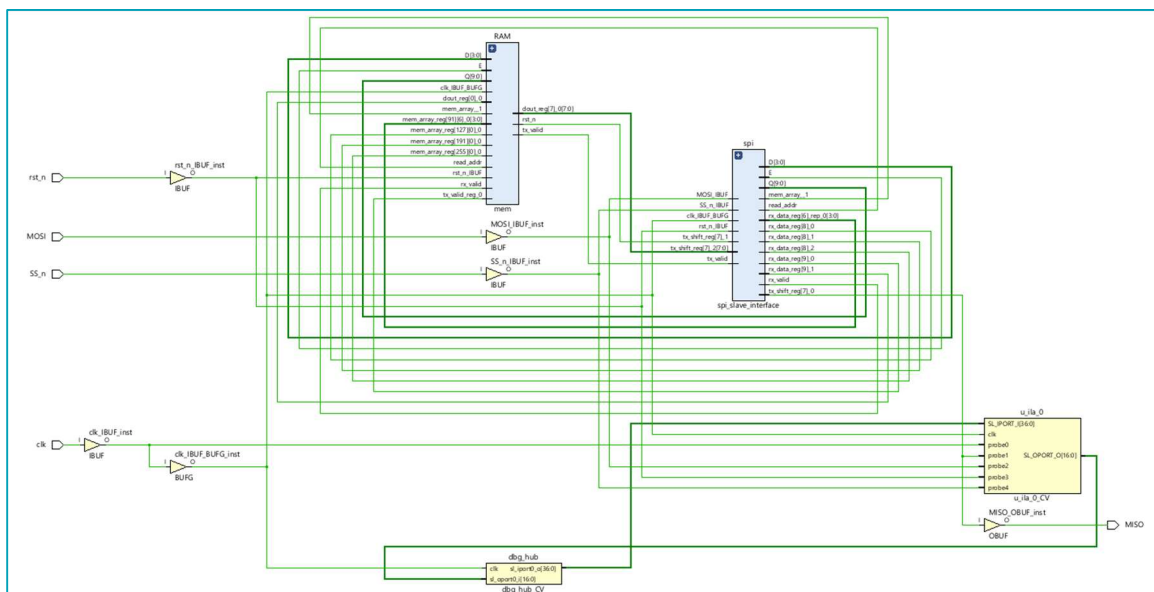


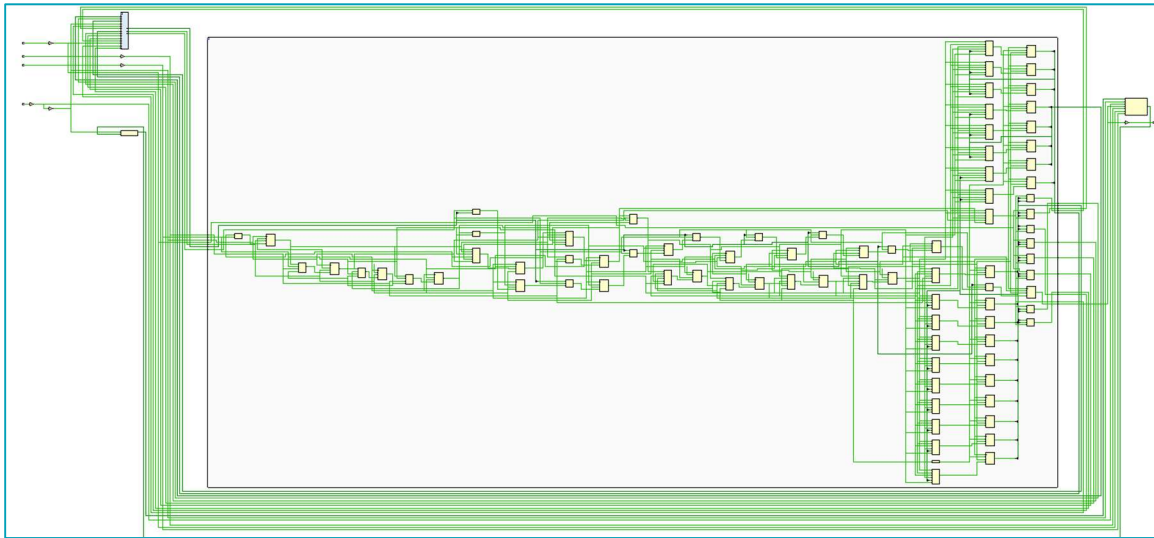
One Hot Encoding

- Post-Elaboration Schematic



- Post-Synthesis Schematic





- Synthesis Report

State	New Encoding	Previous Encoding
IDLE	00001	000
CHK_CMD	00010	001
READ_DATA	00100	011
READ_ADD	01000	010
WRITE	10000	100

INFO: [Synth 8-3354] encoded FSM with state register 'cs_reg' using encoding 'one-hot' in module 'spi_slave_interface'

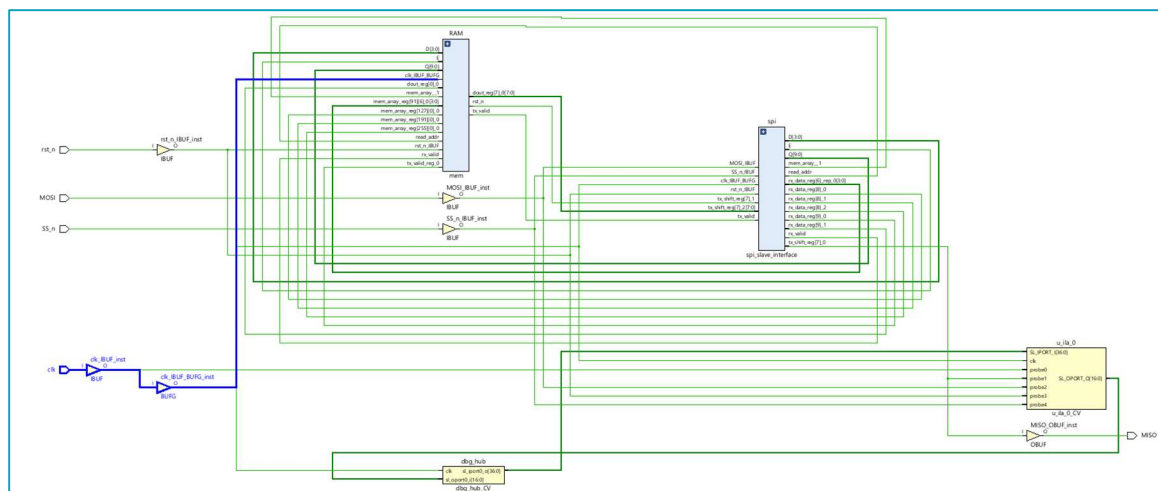
- Post-Synthesis Timing Report

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 5.753 ns		Worst Hold Slack (WHS): 0.134 ns		Worst Pulse Width Slack (WPWS): 4.500 ns	
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 4224		Total Number of Endpoints: 4224		Total Number of Endpoints: 2125	
All user specified timing constraints are met.					

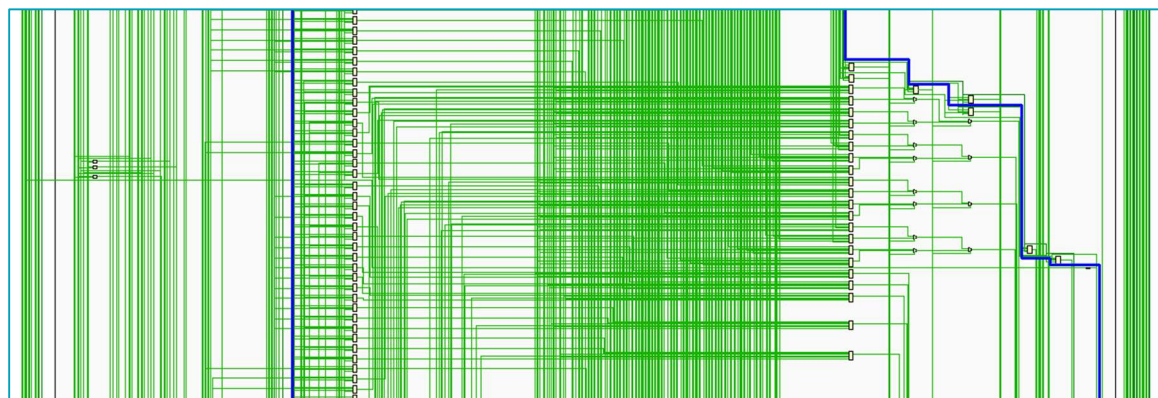
- Critical Path

Critical path is from the clock pin of the memory array register in the RAM module to the data pin of the RAM module, as the Grey encoding case.

Name	Slack	Levels	Routes	High Fanout	From	To	Logic Delay	Total Delay	Net Delay
Path 1	5.753	6	7	1	RAM/mem_array_reg[247][0]/C	RAM/dout_reg[0]/D	1.962	4.096	2.134
Path 2	5.753	6	7	1	RAM/mem_array_reg[247][1]/C	RAM/dout_reg[1]/D	1.962	4.096	2.134
Path 3	5.753	6	7	1	RAM/mem_array_reg[247][2]/C	RAM/dout_reg[2]/D	1.962	4.096	2.134
Path 4	5.753	6	7	1	RAM/mem_array_reg[247][3]/C	RAM/dout_reg[3]/D	1.962	4.096	2.134
Path 5	5.753	6	7	1	RAM/mem_array_reg[247][4]/C	RAM/dout_reg[4]/D	1.962	4.096	2.134
Path 6	5.753	6	7	1	RAM/mem_array_reg[247][5]/C	RAM/dout_reg[5]/D	1.962	4.096	2.134
Path 7	5.753	6	7	1	RAM/mem_array_reg[247][6]/C	RAM/dout_reg[6]/D	1.962	4.096	2.134
Path 8	5.753	6	7	1	RAM/mem_array_reg[247][7]/C	RAM/dout_reg[7]/D	1.962	4.096	2.134
Path 9	6.041	3	4	64	spl/rx_counter_reg[3]/C	RAM/mem_ar...g[0][0]/CE	0.999	3.577	2.578
Path 10	6.041	3	4	64	spl/rx_counter_reg[3]/C	RAM/mem_ar...g[0][1]/CE	0.999	3.577	2.578



- Another Snippet Showing Part of Critical Path After Expanding the RAM:



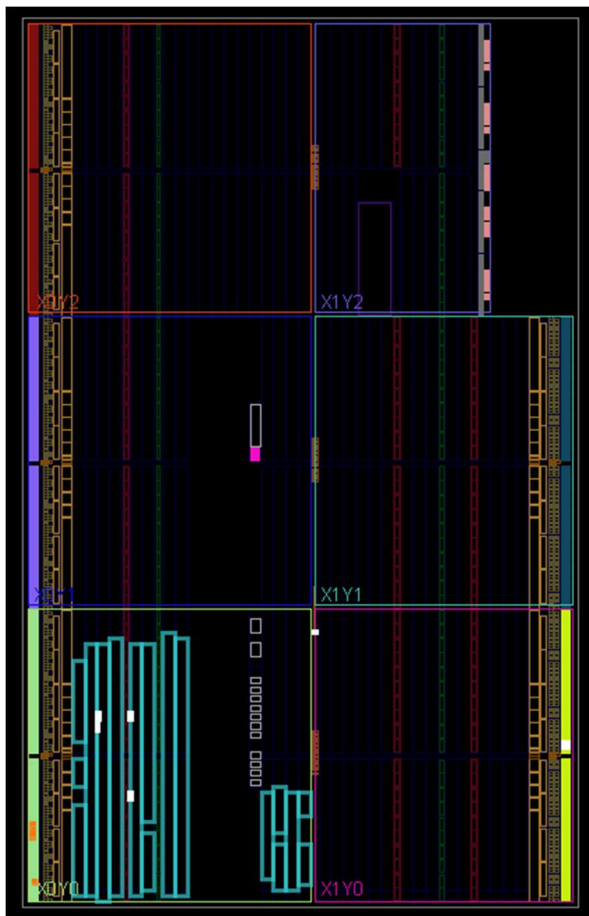
- Post-Implementation Utilization Report

Name	^1	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	F8 Muxes (8150)	Slice (8150)	LUT as Logic (20800)	Bonded IOB (106)	BUFGCTRL (32)
▼ N wrapper		893	2130	272	128	729	893	5	1
RAM (mem)		848	2088	272	128	719	848	0	0
spi (spi_slave_interface)		45	42	0	0	14	45	0	0

- Post-Implementation Timing Report

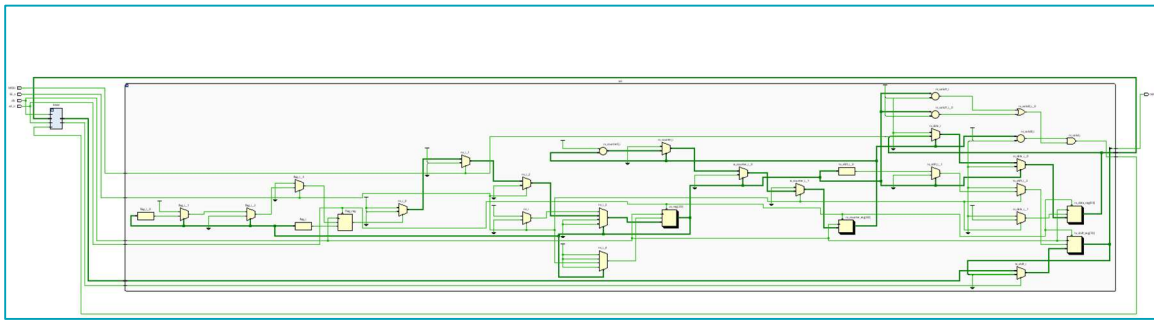
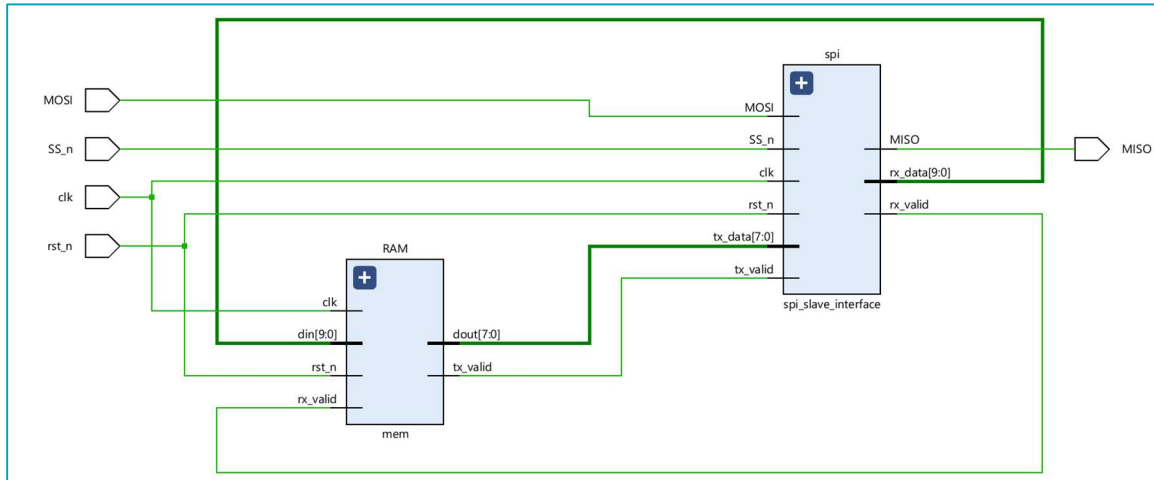
Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 1.147 ns	Worst Hold Slack (WHS): -0.064 ns	Worst Pulse Width Slack (WPWS): 3.750 ns	
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): -0.064 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0	Number of Failing Endpoints: 1	Number of Failing Endpoints: 0	
Total Number of Endpoints: 8024	Total Number of Endpoints: 8008	Total Number of Endpoints: 4249	
Timing constraints are not met.			

- Post-Implementation FPGA Device Snippet

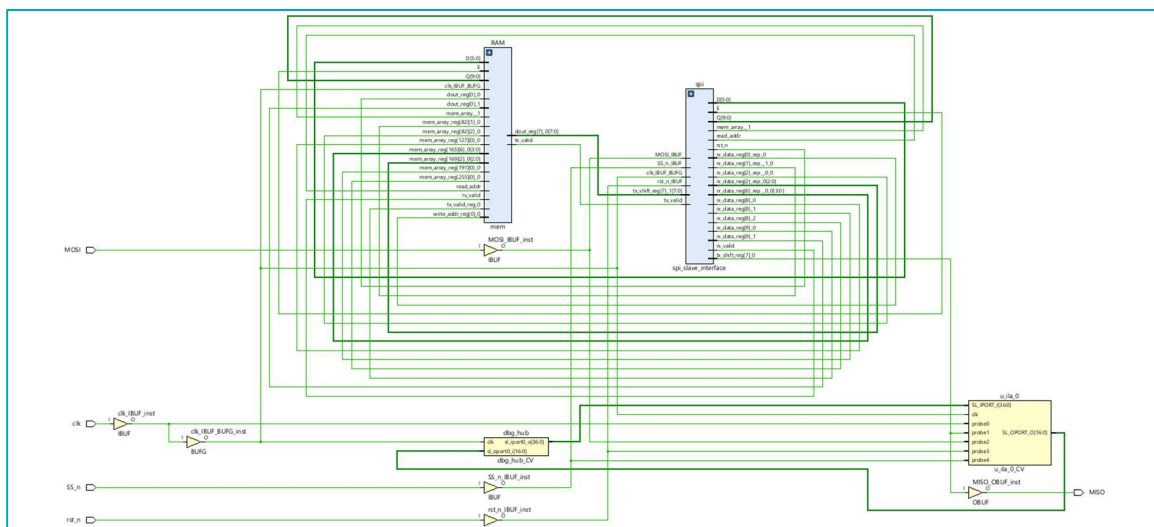


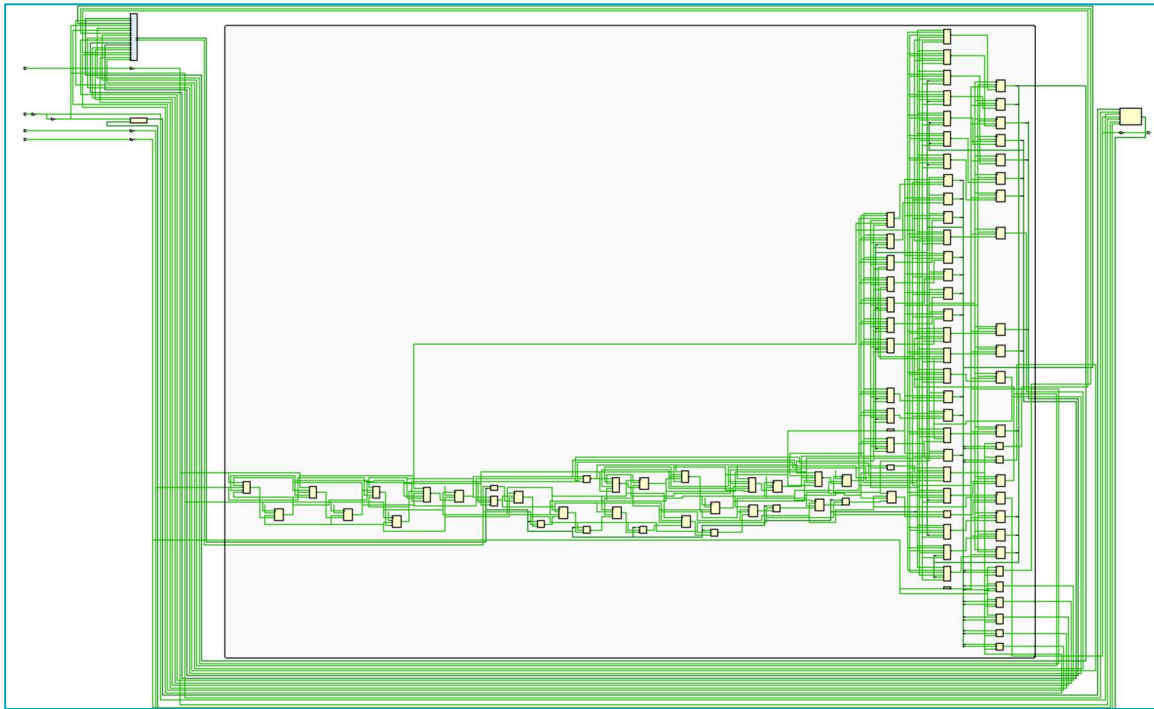
Sequential Encoding

- Post-Elaboration Schematic



- Post-Synthesis Schematic





- Synthesis Report

State	New Encoding	Previous Encoding
IDLE	000	000
CHK_CMD	001	001
READ_DATA	010	011
READ_ADD	011	010
WRITE	100	100

INFO: [Synth 8-3354] encoded FSM with state register 'cs_reg' using encoding 'sequential' in module 'spi_slave_interface'

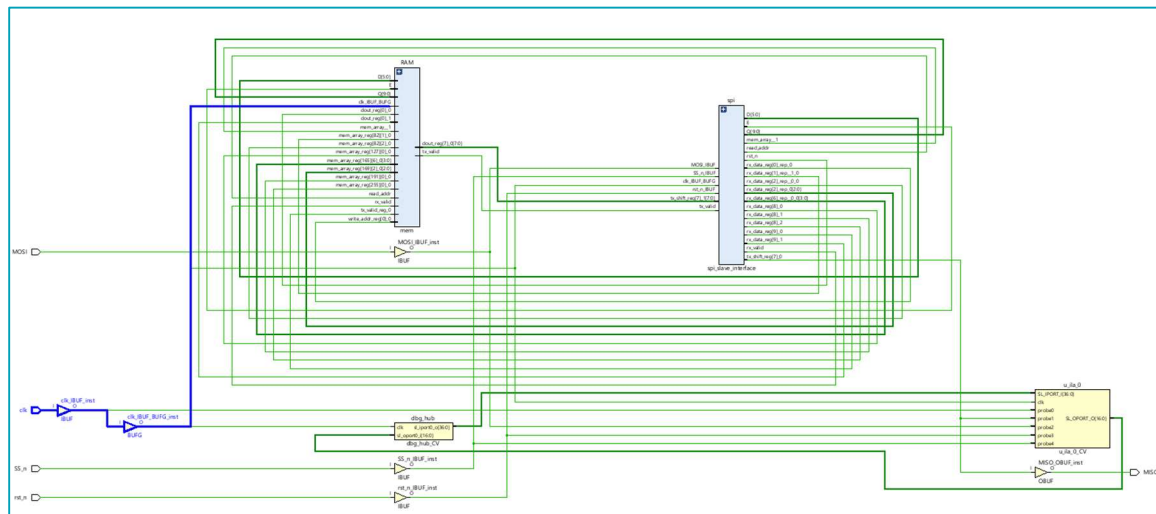
- Post-Synthesis Timing Report

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.753 ns	Worst Hold Slack (WHS): 0.134 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 4225	Total Number of Endpoints: 4225	Total Number of Endpoints: 2131
All user specified timing constraints are met.		

- Critical Path

Critical path is from the clock pin of the memory array register in the RAM module to the data pin of the RAM module, as the Grey encoding case.

Name	Slack	Levels	Routes	High Fanout	From	To	Logic Delay	Total Delay	Net Delay
Path 1	5.753	6	7	1	RAM/mem_arr...[247][0]/C	RAM/dout_reg[0]/D	1.962	4.096	2.134
Path 2	5.753	6	7	1	RAM/mem_arr...[247][1]/C	RAM/dout_reg[1]/D	1.962	4.096	2.134
Path 3	5.753	6	7	1	RAM/mem_arr...[247][2]/C	RAM/dout_reg[2]/D	1.962	4.096	2.134
Path 4	5.753	6	7	1	RAM/mem_arr...[247][3]/C	RAM/dout_reg[3]/D	1.962	4.096	2.134
Path 5	5.753	6	7	1	RAM/mem_arr...[247][4]/C	RAM/dout_reg[4]/D	1.962	4.096	2.134
Path 6	5.753	6	7	1	RAM/mem_arr...[247][5]/C	RAM/dout_reg[5]/D	1.962	4.096	2.134
Path 7	5.753	6	7	1	RAM/mem_arr...[247][6]/C	RAM/dout_reg[6]/D	1.962	4.096	2.134
Path 8	5.753	6	7	1	RAM/mem_arr...[247][7]/C	RAM/dout_reg[7]/D	1.962	4.096	2.134
Path 9	5.929	4	5	64	spi/rx_counter_reg[2]/C	RAM/mem_ar...g[0][0]/CE	1.149	3.689	2.540
Path 10	5.929	4	5	64	spi/rx_counter_reg[2]/C	RAM/mem_ar...g[0][1]/CE	1.149	3.689	2.540



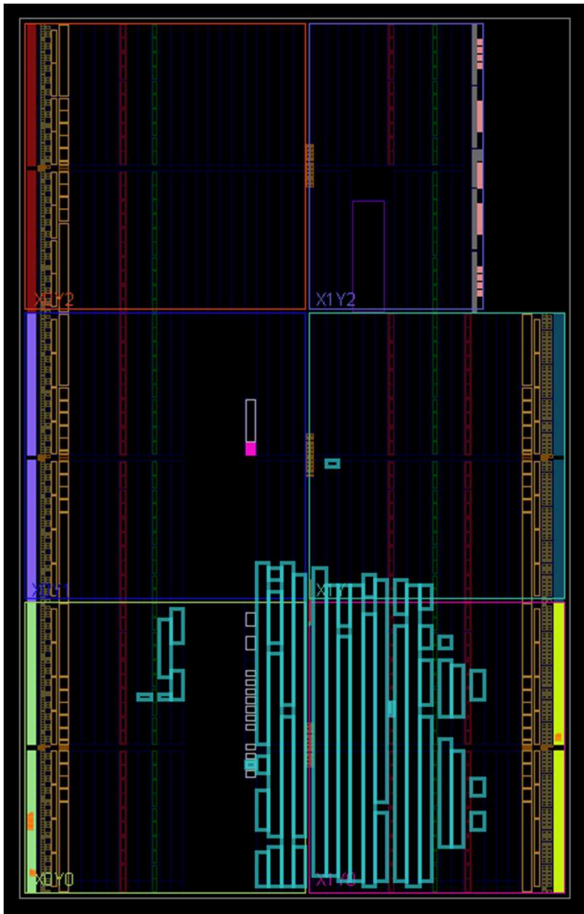
- Post-Implementation Utilization Report

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	F8 Muxes (8150)	Slice (8150)	LUT as Logic (20800)	LUT as Memory (9600)	Block RAM Tile (50)	Bonded IOB (106)	BUFGCTRL (32)	BSCAN2 (4)
▼ N wrapper	2101	4070	275	128	1324	1993	108	0.5	5	2	1
> dbg_hub (dbg_hub)	449	741	0	0	228	425	24	0	0	1	1
RAM (mem)	848	2088	272	128	726	848	0	0	0	0	0
spi (spi_slave_interface)	45	42	0	0	17	45	0	0	0	0	0
> u_ila_0 (u_ila_0)	759	1199	3	0	370	675	84	0.5	0	0	0

- Post-Implementation Timing Report

Design Timing Summary			
Setup	Hold		Pulse Width
Worst Negative Slack (WNS): 2.061 ns	Worst Hold Slack (WHS): 0.036 ns	Worst Pulse Width Slack (WPWS): 3.750 ns	
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	
Total Number of Endpoints: 8025	Total Number of Endpoints: 8009	Total Number of Endpoints: 4255	
All user specified timing constraints are met.			

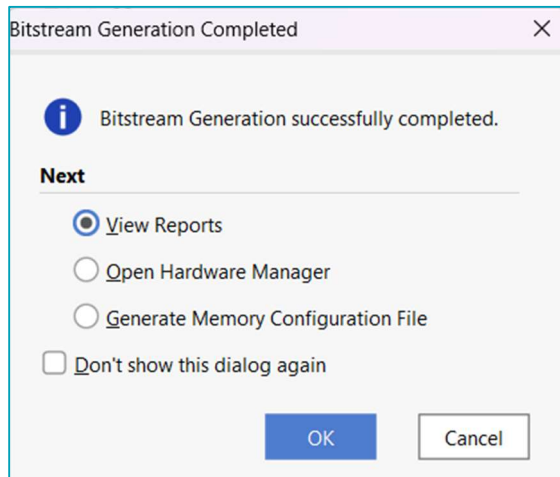
- Post-Implementation FPGA Device Snippet



Optimal Choice

In order to achieve the maximum frequency, one should use the encoding that achieves the maximum slack. In this design, the encoding that achieved this was Gray encoding.

Bitstream Generation



Messages Tab

